

Atividade_08 - Livro AVR e Arduino – Técnicas de Projeto

Capítulo: 15 (USART)

Título: Usando a porta serial para controlar dispositivos

Objetivos: Aprender a usar a porta serial dos microcontroladores da Atmel. Aprender a usar a porta serial dirigida a interrupções.

Nesta prática utilizaremos o Tinkercad para simular um circuito simples usando o microcontrolador Atmega328, utilizado nas placas Arduino UNO. Desta vez, programaremos usando um código C para acender e apagar LEDs usando a porta serial. Faremos também toda a comunicação ser dirigida a interrupções.

1. Procedimentos:

1. Acesse sua conta no Tinkercad ([tinkercad.com](https://www.tinkercad.com)) e vá para a aba circuits (<https://www.tinkercad.com/circuits>). Você pode utilizar o projeto disponível em https://www.tinkercad.com/things/eyLhoLGx4fb?sharecode=E-6BB7x5jBR_8XXzy-t6JD5IKrhkZ1oBAhds1yoqso como exemplo de como configurar uma porta serial que utiliza interrupções para envio não bloqueante e https://www.tinkercad.com/things/fRn4CMurk9T?sharecode=7Nj0AdwO1J6dBTEINEJiLKQdIgPpUrhvkZO_gVTddBU como exemplo de recebimento não bloqueante.

2. Ligue dois LEDs, um na porta PB5 (13), um na porta PB4 (12).

3. Agora você deve fazer com que o LED na porta PB5 (13) acenda ao se enviar 'A13' e apague ao se enviar 'S13'. O LED na porta PB4 (12) deve alternar de estado toda vez que 'D12' for recebido.

4. Agora você deve modificar o código de forma a enviar um texto pela porta serial de tempos em tempos. Este código deve rodar dentro do while() no main(). O texto será 10000+(millis())%90000). Ele deve ser enviado em intervalos de 50ms e 10ms alternadamente.

Utilize a função init() para que millis() funcione.

Modelo de função principal:

```
int main() {  
  init();  
  configura_serial(2400);  
  while(1) {  
    if (passou 50ms)  
      envia_millis(); // não bloqueante (millis e quebra de linha)  
    if (passou 10ms)  
      envia_millis(); // não bloqueante (millis e quebra de linha)  
    if (dados recebidos na serial)  
      atua nos leds correspondentes  
  }  
}
```

5. Todo o código da porta serial deve ser dirigido a interrupções. A velocidade de transmissão da serial deve ser **2400bps**.

6. Cole o código fonte do microcontrolador ao final deste arquivo e inclua a imagem de seu design. Importante: Deixe seu circuito compartilhado por link no Tinkercad e cole o link para ele aqui:

<https://www.tinkercad.com/things/4phLoXMbgqk-copy-of-usarttxasynccircbuff?sharecode=EyYRyZw1MotoOykqdUk2Wqnit4gDGd6gyuLb2-WWm08>

ATENÇÃO: Documente seu código. Cada linha/bloco deve deixar explícito o seu papel.

ATENÇÃO: Na versão final do seu projeto, as funções `pinMode()`, `digitalWrite()` e `digitalRead()` são proibidas. O uso delas fará a nota atribuída ser zero. **Utilizar USART com envio e recebimento dirigido a interrupções é obrigatório.**

Proibido utilizar classes prontas que encapsulem funcionalidades da porta serial, como a classe Serial do Arduino. A atividade somente será avaliada com um link válido para o projeto.

Obrigatório para a avaliação: Enviar também o código fonte em um arquivo separado.

Sempre que julgar necessário, o professor pode solicitar a apresentação da atividade para auxiliar no processo de correção.

RÚBRICA:

10% Circuito;

40% LEDs funcionam como descrito (são ativados via serial sem delays);

50% O valor do `millis()` é enviado no tempo certo sem afetar as demais funções;

Valor desta atividade na média: 0.7

```
#define F_CPU 16000000UL // Define a frequência do microcontrolador - 16MHz
```

```
#include <avr/io.h> // Definições de registradores do componente (AVR)
```

```
#include <util/delay.h> // Biblioteca para rotinas de delay
```

```
#include <avr/pgmspace.h> // Gravação de dados na memória flash
```

```
#include <avr/interrupt.h> // Definições de interrupções (sei, cli, ISR)
```

```
#include <stdio.h> // Para uso do snprintf
```

```
#include <string.h> // Para uso do memcpy
```

```
#include <Arduino.h> // Necessário para as funções init() e millis()
```

```
// EXIGÊNCIA 5: Taxa de transmissão configurada obrigatoriamente para 2400 bps
```

```
#define BAUD 2400
```

```
#define MYUBRR F_CPU/16/BAUD-1
```

```
// Aumentado para 64 para garantir estabilidade e evitar estouro durante o envio rápido de 10ms
```

```
#define BUFFLEN 64
```

```
char buff[BUFFLEN]; // Buffer circular de envio (TX)
```

```
// Variáveis de controle do buffer de envio
```

```
short pos=0, end=0, cntbuff=0;
```

```
// ETAPA 3: Definições de tamanho e variáveis para o Buffer Circular de Recepção (RX)
```

```
#define RX_BUFFLEN 32
```

```
volatile char rx_buff[RX_BUFFLEN]; // Buffer circular de recebimento (RX)
```

```
volatile short rx_pos=0, rx_end=0, rx_cnt=0; // Variáveis de controle do RX de forma volátil
```

```
//-----
```

```
void USART_Inic(unsigned int ubrr0)
```

```
{
```

```
    UBRR0H = (unsigned char)(ubrr0>>8); // Ajusta a taxa de transmissão (Byte Alto)
```

```
    UBRR0L = (unsigned char)ubrr0; // Ajusta a taxa de transmissão (Byte Baixo)
```

```
    UCSR0A = 0; // Desabilita velocidade dupla (padrão do Arduino)
```

```
// Habilita Transmissão (TXEN0), Recepção (RXEN0) e a Interrupção de Recepção Completa (RXCIE0)
```

```
UCSR0B = (1<<RXCIE0)|(1<<RXEN0)|(1<<TXEN0);
```

```
// Modo assíncrono, 8 bits de dados, 1 bit de parada, sem paridade
```

```
UCSR0C = (1<<UCSZ01)|(1<<UCSZ00);
```

```
sei(); // Habilita globalmente as interrupções  
}
```

```
//-----
```

```
void EscreveAsync(char *dados, short len) {
```

```
// Desabilita temporariamente a interrupção de registrador vazio do TX para consistência
```

```
UCSR0B &= ~(1<<UDRIE0);
```

```
// Proteção básica para garantir que o tamanho não estoure o tamanho do buffer  
len %= BUFFLEN;
```

```
cntbuff += len;
```

```
short cnt = BUFFLEN-end;
```

```
cnt = cnt>len?len:cnt;
```

```
memcpy(buff+end, dados, cnt);
```

```
if (cnt<len)
```

```
    memcpy(buff, dados+cnt, len-cnt);
```

```
end += len;
```

```
end %= BUFFLEN;
```

```
// Reabilita a interrupção do TX para iniciar/continuar o envio em background
```

```
UCSR0B |= (1<<UDRIE0);
```

```
}
```

```
//-----
```

```
// ISR de Transmissão: Dispara sempre que o registrador de dados da USART esvazia
```

```
ISR(USART_UDRE_vect) {
```

```
    if (cntbuff>0) {
```

```
        UDR0 = buff[pos]; // Envia o caractere atual do buffer
```

```
        pos++;
```

```
        cntbuff--;
```

```
        pos %= BUFFLEN; // Incremento circular do ponteiro
```

```
    } else {
```

```
        // Se não houver mais dados para enviar, desliga a própria interrupção
```

```
        UCSR0B &= ~(1<<UDRIE0);
```

```
    }
```

```
}
```

```
//-----
```

```
// ETAPA 3: Implementação da ISR de Recepção (RX) dirigida por interrupção
```

```
ISR(USART_RX_vect) {
```

```
    char c = UDR0; // Captura o caractere vindo diretamente do registrador físico do hardware
```

```
// Armazena no buffer de recepção apenas se ele não estiver completamente cheio
```

```
if (rx_cnt < RX_BUFFLEN) {
```

```
    rx_buff[rx_end] = c;          // Guarda o caractere recebido
```

```

    rx_end = (rx_end + 1) % RX_BUFFLEN; // Avança o ponteiro de fim do buffer
circular
    rx_cnt++; // Incrementa contador de dados armazenados
}
}

//-----
// ETAPA 4: Função auxiliar para formatar e despachar o millis de forma assíncrona
void envia_millis() {
    char texto_formatado[16];

    // Calcula a fórmula solicitada: 10000 + (millis() % 90000)
    uint32_t resultado = 10000 + (millis() % 90000);

    // Formata o número em string adicionando a quebra de linha (\n)
    short tamanho = sprintf(texto_formatado, sizeof(texto_formatado), "%lu\n",
    resultado);

    // Envia a string formatada usando a rotina não-bloqueante
    EscreveAsync(texto_formatado, tamanho);
}

//-----
int main()
{
    // ETAPA 4: Inicializa temporizadores do hardware Arduino para o millis()
funcionar
    init();

    // Inicializa o periférico USART com base em interrupções (2400 bps)
    USART_Inic(MYUBRR);

    // PROIBIDO PINMODE: Configura diretamente os pinos PB5 (D13) e PB4 (D12)
como saídas
    DDRB |= (1 << DDB5) | (1 << DDB4);

    // PROIBIDO DIGITALWRITE: Garante que ambos os LEDs comecem totalmente
apagados
    PORTB &= ~(1 << PORTB5) | (1 << PORTB4));

    // Variáveis para gerenciar o tempo compartilhado de forma não-bloqueante
    unsigned long tempo_anterior = millis();
    unsigned char alternador_intervalo = 1; // Controla se o próximo passo aguardará
50ms (1) ou 10ms (0)

    // Janela estática de 3 posições para monitorar strings de comando de forma
contínua
    static char janela_comandos[3] = {0, 0, 0};

    while(1) // Laço principal perpétuo (não-bloqueante)
    {
        unsigned long tempo_atual = millis();

        // ETAPA 4: Lógica alternada de verificação de tempo (50ms)
        if (alternador_intervalo == 1 && (tempo_atual - tempo_anterior >= 50)) {
            envia_millis(); // Executa rotina não-bloqueante
            tempo_anterior = tempo_atual; // Atualiza marca de tempo
            alternador_intervalo = 0; // Altera o estado do flag para esperar 10ms no
próximo ciclo

```

```

}
// ETAPA 4: Lógica alternada de verificação de tempo (10ms)
else if (alternador_intervalo == 0 && (tempo_atual - tempo_anterior >= 10)) {
    envia_millis(); // Executa rotina não-bloqueante
    tempo_anterior = tempo_atual; // Atualiza marca de tempo
    alternador_intervalo = 1; // Retorna o flag para esperar 50ms na próxima
vez
}

// ETAPA 3 & 4: Verifica se há dados pendentes recebidos na serial (rx_cnt > 0)
if (rx_cnt > 0) {
    char c = rx_buff[rx_pos]; // Lê o caractere armazenado na posição atual
    rx_pos = (rx_pos + 1) % RX_BUFFLEN; // Avança o ponteiro circular de leitura

    // Seção Crítica: desabilita interrupção rapidamente para decrementar o
    contador de forma segura
    cli();
    rx_cnt--;
    sei();

    // Desloca a janela de detecção para a esquerda para acomodar o novo caractere
    na ponta direita
    janela_comandos[0] = janela_comandos[1];
    janela_comandos[1] = janela_comandos[2];
    janela_comandos[2] = c; // Novo dado

    // ETAPA 3: Atuação nos LEDs baseada no conteúdo da janela usando
    manipulação direta de registradores

    // Se detectado 'A13' -> Liga o LED conectado na porta PB5
    if (janela_comandos[0] == 'A' && janela_comandos[1] == '1' &&
janela_comandos[2] == '3') {
        PORTB |= (1 << PORTB5); // Aplica nível alto em PB5 (Bit Set)
    }
    // Se detectado 'S13' -> Apaga o LED conectado na porta PB5
    else if (janela_comandos[0] == 'S' && janela_comandos[1] == '1' &&
janela_comandos[2] == '3') {
        PORTB &= ~(1 << PORTB5); // Aplica nível baixo em PB5 (Bit Clear)
    }
    // Se detectado 'D12' -> Alterna o estado do LED conectado na porta PB4
    else if (janela_comandos[0] == 'D' && janela_comandos[1] == '1' &&
janela_comandos[2] == '2') {
        PORTB ^= (1 << PORTB4); // Inverte o estado atual do bit PB4 (Operação
XOR)
    }
}
}
return 0;
}

```